CHAPTER 11

Securing the API with JWT

Authentication for stateless RESTful services

SCSM2223 — Cross-Platform Application Development
Faculty of Computing • Universiti Teknologi Malaysia
Assoc. Prof. Dr Hishammuddin bin Asmuni

AT THE END OF THIS CHAPTER

Learning Objectives

C1	State what authentication and authorization are, and how they differ.
C2	Differentiate session-based from token-based authentication and explain when to use each.
C2	Describe the structure of a JWT (header, payload, signature) and the role of the signing key.
C3	Hash passwords with password_hash and verify them with password_verify.
C3	Issue and verify JWTs in PHP using the firebase/php-jwt library.
C3	Build an AuthMiddleware that protects selected routes in a Slim API.

Where We Are

CHAPTER 10 — Today

- ✓ REST API with PDO + MySQL
- ✓ Prepared statements
- ✓ Repository pattern
- ✗ **Anyone can hit any endpoint**
- ✗ **No way to identify the caller**
- ✗ **No protection on POST/PUT/DELETE**

CHAPTER 11 — Goal

- ✓ Users can register and log in
- ✓ Passwords stored as hashes
- ✓ Server issues a signed JWT
- ✓ Clients send token on every call
- ✓ Middleware verifies the token
- ✓ Stateless — no server-side session

TWO WORDS THAT SOUND ALIKE

Authentication vs Authorization

Authentication — AuthN

Who are you?

- Verifies the identity of the caller.
- Common factors:
  - something you know (password)
  - something you have (phone, key)
  - something you are (biometric)
- Result: a verified user identity.

Authorization — AuthZ

What are you allowed to do?

- Decides whether an authenticated user may perform an action.
- Driven by ROLES (admin, editor, member) or POLICIES (RBAC, ABAC).
- Examples:
  - any user can read books
  - only admins can delete books

TWO APPROACHES

Session-based vs Token-based Authentication

ASPECT	Session-based	Token-based (JWT)
State	Server holds session in memory / Redis / DB	Stateless — token holds the data
Sent in	Cookie (sticky session)	Authorization: Bearer <token>
Scaling	Need shared session store across servers	Any server can validate independently
Mobile / SPA	Workable but cookie-heavy	Natural fit for fetch / Axios
Logout	Easy (delete server entry)	Needs a denylist or short TTL
Best for	Traditional server-rendered pages	REST APIs, mobile apps, microservices

This course uses JWT — it suits our Vue + Slim REST architecture.

What is a JSON Web Token?

A compact, URL-safe, digitally-signed JSON object that proves a claim about a user.

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOjQyLCJlbWFpbCI6ImhpbGFuZ0BleGFtcGxlLmNvbSIsImlhdCI6MTcxNzAwMDAwMCwi
ZXhwIjoxNzE3MDAzNjAwLCJyb2xlIjoiYWRtaW4ifQ.
S9-2pFJOYpvlcK9b0Q-EItDsqOqSkmwdGdqZ7N8a4Vc
```

Header

base64url-encoded JSON describing the algorithm and token type.

```
{ "alg": "HS256", "typ": "JWT" }
```

Payload

base64url-encoded JSON of the claims (sub, iat, exp, role…).

```
{ "sub": 42, "role": "admin", "exp": 1717003600 }
```

Signature

HMAC of base64(header) + "." + base64(payload), keyed with the server secret.

Verifies the token has NOT been tampered with.

WHAT IS INSIDE

A JWT, Decoded

HEADER → base64url decode

```
{
  "alg": "HS256",   // signing algorithm
  "typ": "JWT"
}
```

PAYLOAD → base64url decode

```
{
  "sub":   42,            // subject (user id)
  "email": "hilang@example.com",
  "role":  "admin",
  "iat":   1717000000,    // issued at
  "exp":   1717003600   // expires (1 hour later)
}
```

SIGNATURE — verify with the same secret + algorithm

```
HMACSHA256(
  base64UrlEncode(header) + "." +
base64UrlEncode(payload),
  YOUR_SERVER_SECRET
)
```

⚠ ***Anyone can READ the payload — never put passwords or secrets in there.***

HS256 vs RS256 — Pick One

HS256 (HMAC + SHA-256)

Symmetric — same secret to sign and verify.

- Simple. One secret string in your .env.
- Both issuer and verifier must hold it.
- Best for monolithic apps where the same backend issues AND verifies.
- **What we use in this course.**

RS256 (RSA + SHA-256)

Asymmetric — private key signs, public key verifies.

- Issuer keeps the private key.
- Anyone with the public key can verify.
- Useful when one server issues tokens that many other services verify (microservices, OIDC).
- More setup; not needed in this course.

END-TO-END FLOW

JWT Lifecycle

1	**POST /auth/login**	Client sends email + password.
2	**Verify credentials**	Server uses password_verify on the stored hash.
3	**Issue token**	Server signs a JWT with sub, exp, role, then returns it.
4	**Client stores token**	In memory or in localStorage / cookie.
5	**Send on every call**	Authorization: Bearer <token> header.
6	**Middleware verifies**	Decodes, checks signature + expiry, exposes user info.
7	**Token expires**	Client logs in again or uses a refresh token.

STEP 1 — STORE SAFELY

Hashing Passwords with password_hash

Never store plain passwords. Hash on registration; verify on login.

```
// Registration — store the hash, never the password
$hash = password_hash($plainPassword, PASSWORD_DEFAULT);
// → looks like: $2y$12$Bd0nFv5Jj78A4oLtYV4VYeR.OJwK0c7M2kPSc5w...

// Login — verify what the user sent against the stored hash
if (password_verify($plainPassword, $row['password_hash'])) {
    // success — issue a JWT
} else {
    // 401 Unauthorized — credentials invalid
}

// Bonus: rehash if PHP's default cost has increased
if (password_needs_rehash($row['password_hash'], PASSWORD_DEFAULT)) {
    $newHash = password_hash($plainPassword, PASSWORD_DEFAULT);
    // store $newHash
}
```

Login Endpoint — Issuing a JWT

```
use Firebase\JWT\JWT;

$app->post('/auth/login', function (Request $r, Response $s) use ($pdo) {
    $body = (array)$r->getParsedBody();
    $stmt = $pdo->prepare('SELECT * FROM users WHERE email = :e');
    $stmt->execute([':e' => $body['email'] ?? '']);
    $user = $stmt->fetch();

    if (!$user || !password_verify($body['password'] ?? '', $user['password_hash'])) {
        $s->getBody()->write(json_encode(['error' => 'Invalid credentials']));
        return $s->withStatus(401)->withHeader('Content-Type','application/json');
    }

    $now = time();
    $payload = [
        'iss'  => 'books-api',
        'sub'  => (int)$user['id'],
        'role' => $user['role'],
        'iat'  => $now,
        'exp'  => $now + 3600,        // 1-hour TTL
    ];
    $token = JWT::encode($payload, $_ENV['JWT_SECRET'], 'HS256');
    $s->getBody()->write(json_encode(['token' => $token]));
    return $s->withHeader('Content-Type','application/json');
});
```

AuthMiddleware — Verifying a JWT

```
use Firebase\JWT\JWT;
use Firebase\JWT\Key;

final class AuthMiddleware implements MiddlewareInterface
{
    public function process($request, $handler): ResponseInterface
    {
        $auth = $request->getHeaderLine('Authorization');     // 'Bearer eyJ…'
        if (!preg_match('/^Bearer\s+(.+)$/', $auth, $m)) {
            return $this->unauthorised('Missing token');
        }
        try {
            $decoded = JWT::decode($m[1], new Key($_ENV['JWT_SECRET'], 'HS256'));
        } catch (\Throwable $e) {
            return $this->unauthorised('Invalid or expired token');
        }
        // Attach the decoded user info onto the request for handlers to use.
        $request = $request->withAttribute('auth', (array)$decoded);
        return $handler->handle($request);
    }
    private function unauthorised(string $msg): ResponseInterface { /* 401 JSON */ }
}
```

Protecting Selected Routes

```
$auth = new AuthMiddleware();

// Public routes — anyone can call
$app->post('/auth/register', [AuthController::class, 'register']);
$app->post('/auth/login',    [AuthController::class, 'login']);
$app->get('/api/books',      [BookController::class, 'index']);
$app->get('/api/books/{id}', [BookController::class, 'show']);

// Protected routes — must present a valid JWT
$app->group('/api/books', function ($g) use ($controller) {
    $g->post  ('',      [$controller, 'create']);
    $g->put   ('/{id}', [$controller, 'update']);
    $g->delete('/{id}', [$controller, 'delete']);
})->add($auth);

// Inside any handler that runs after $auth, you can read:
// $req->getAttribute('auth')['sub']   ← user id
// $req->getAttribute('auth')['role']  ← role
```

FRONT-END CONCERN

Where Should the Browser Store the Token?

OPTION	Pros	Cons
In-memory (JS variable)	Survives no XSS attacks because it lives only in script	Lost on every refresh
localStorage / sessionStorage	Easy to read with fetch / Axios	Vulnerable to XSS — any injected script can steal it
Cookie (HttpOnly + Secure + SameSite)	Cannot be read by JS; XSS-safe	Needs CSRF protection on state-changing routes

Pragmatic course choice: localStorage during development, with strong XSS hygiene (Vue auto-escapes by default).

Refresh Tokens — When the Access Token Expires

Short-lived access token + longer-lived refresh token. The refresh token is exchanged for a fresh access token without forcing the user to log in again.

Access token	Short TTL (e.g. 15 min). Sent in Authorization header on every API call.
Refresh token	Long TTL (days/weeks). Stored as HttpOnly cookie or in mobile secure storage.
Rotation	Each refresh issues a new pair; the old refresh token is revoked.
Revocation	Maintain a denylist (e.g. by jti claim) to log users out before expiry.

Out of scope for the lab — but a great extension task.

DON'T DO THESE

Common JWT Pitfalls

Hard-coded secret in source
Read JWT_SECRET from .env; never commit it.

alg=none accepted
Always pin the algorithm in JWT::decode (e.g. HS256).

No exp claim
Tokens that never expire are tickets to forever-compromise.

Sensitive data in payload
Anyone can decode it. No passwords, no PII.

Short access TTL + refresh
Limit blast radius if a token leaks.

HTTPS everywhere
Bearer tokens travel in plain text otherwise.

Summary — Chapter 11

AuthN vs AuthZ	Identity vs permission — two distinct concerns.
Tokens	Stateless JWTs replace server sessions for REST APIs.
Structure	JWT = base64(header).base64(payload).signature.
Hashing	Always password_hash on register, password_verify on login.
Middleware	Verify token in middleware; route handlers stay clean.

Next Up

Lab Hands-On — JWT for the Books API

Add user registration, login, and JWT-protected endpoints to the Chapter 10 Books API.

- Add a users table and seed data
- Install firebase/php-jwt
- POST /auth/register with password_hash
- POST /auth/login → issues a JWT
- AuthMiddleware verifies tokens
- Protect POST/PUT/DELETE on /api/books
- Test the full flow end-to-end